Broadcasting Multiple Messages in Simultaneous Send/Receive Systems

Amotz Bar-Noy

IBM T. J. Watson Research Center
Yorktown Heights, NY

Shlomo Kipnis

IBM T. J. Watson Research Center
Yorktown Heights, NY

Abstract

We investigate the problem of broadcasting multiple messages in a message-passing system that supports simultaneous send and receive. The system consists of n processors, one of which has m messages to broadcast to the other $n-1$ processors. The processors communicate in rounds. In each round, a processor can simultaneously send a message to one processor and receive a message from another processor. The goal is to broadcast the m messages among the n processors in the minimal number of communication rounds. The lower bound on the number of rounds required is $(m-1)+\lceil \log n \rceil$. We present an algorithm for this problem that requires at most $m+\lceil \log n \rceil$ communication rounds, for any values of m and n.

1 Introduction

Broadcasting is one of the most important communication operations in many multi-processor systems. Application areas that use this operation extensively include scientific computations, database transactions, network management protocols, and multimedia applications. Due to the significance of the broadcasting operation, it is important to design efficient algorithms for implementing it.

Several variations of the broadcasting problem were studied in the parallel and distributed processing literature. Most of this research focuses on designing broadcasting algorithms for specific network topologies such as rings, trees, meshes, and hypercubes. However, an emerging trend in many message-passing systems is to treat the system as a fully-connected collection of processors in which each pair of processors can communicate directly. This strategy is used, for example, in several distributed-memory parallel computers [3, 8, 11] and in some high-speed communication networks [5, 6].

When broadcasting large amounts of data, many systems break the data into sequences of messages that are broadcast individually. The problem of broadcasting multiple messages in fully-connected systems was studied in several communication models. The papers [7, 9] present optimal-time solutions for this problem in a model where each processor can either send or receive one message in any communication round, but not both. The papers [1, 2] investigate the problem of broadcasting in the postal model of communication, in which each processor can concurrently send one message and receive another message but message delivery involves communication latency.

This paper investigates the problem of broadcasting multiple messages in a fully-connected message-passing system with simultaneous send and receive. The system consists of n processors, one of which has m messages to broadcast to the other processors. The processors communicate in rounds. In each round, a processor can simultaneously send a message to one processor and receive a message from another processor. The goal is to broadcast the m messages among the n processors in the minimal number of rounds. A lower bound on the number of rounds required is $(m-1)+\lceil \log n \rceil$. However, no algorithm is known that matches this bound for all values of m and n.

Optimal solutions for the problem of broadcasting multiple messages in a system that supports simultaneous send and receive are known only for specific values of m and n. In the case of $m=1$ and for any value of n, a simple folklore $\lceil \log n \rceil$-rounds algorithm based on recursive doubling provides an optimal solution for this problem. When $n=2^k$ and for any value of m, [10] provides an optimal $(m-1)+\log n$-rounds algorithm based on disjoint spanning trees in a binary hypercube. However, this solution relies heavily on the fact that n is a power of 2 and cannot be used for other values of n. For general values of m and n, a special case of one of the algorithms in [2] provides a solution to the broadcasting problem considered here. The running time of this solution is $m+2\log n+O(1)$ rounds. Finally, the best known broadcasting algorithm for arbitrary values of m and n was recently developed in [4]. The running time of this algorithm is $m+\log n+O(\log\log n)$.

This paper provides an optimal-time algorithm (to within an additive term of 2) for this problem. The algorithm presented here is practical and works for any values of m and n. The running time of the algorithm is at most $m+\lceil \log n \rceil+1$ rounds. With some minor modificattions to this algorithm, we can obtain an algorithm the running time of which is $m+\lceil \log n \rceil$. Details are omitted from this abstract.

1063-6374/93 $03.00 © 1993 IEEE

2 General idea of the algorithm

The broadcasting algorithm uses a simple folklore algorithm for broadcasting $m = 1$ messages among $n = 2^k$ processors in $k = \log n$ rounds. This algorithm consists of doubling the number of processors that know the message every communication round. We call this algorithm Algorithm $\mathcal{A}$.

Another well-known algorithm broadcasts $m > 1$ messages among $n = 2^k$ processors in $m+k = m+\log n$ rounds. In this algorithm, the $n = 2^k$ processors are arranged as a k-dimensional hypercube. The broadcast source (root) sends the m messages one after the other in a cyclic order to $k = \log n$ designated processors. Each of these k processors applies Algorithm $\mathcal{A}$ among all the n processors, thereby acting as a broadcast originator for each message it receives from the root. In each round of this algorithm, messages are exchanged only along one of the k hypercube dimensions. We call this algorithm Algorithm $\mathcal{B}$.

In Algorithm $\mathcal{B}$, each of the k designated processors returns to the root the last message it received from it in the same round in which the root sends a new message to that processor. This step is redundant and can be replaced by having each of the k designated processors, in turn, send its message outside the hypercube of the 2^k processors. In doing so, Algorithm $\mathcal{B}$ can be viewed as a "black-box" with the following properties: (i) the black-box contains $2^k - 1$ processors; (ii) a stream of m messages from some external source is injected into the black-box; (iii) each message in the stream becomes known to all the processors in the black-box exactly k rounds after it is injected into the black box; and (iv) the stream of m messages is ejected from the black-box exactly k rounds after it is injected into it.

A broadcasting algorithm for any values of m and n can be constructed by concatenating a sequence of Algorithm $\mathcal{B}$'s black-boxes that include all the $n-1$ recipient processors. The output stream of one black-box in the sequence serves as the input stream of the next black-box. The input stream of the first black-box is provided by the broadcast source and the output stream of the last black-box is discarded. However, such a solution requires $m+O(\log^2 n)$ rounds. This is because the output stream of a black-box with $2^k - 1$ processors is delayed k rounds after its input stream, and the sum of the delays of all the black boxes in the sequence may be as high as $O(\log^2 n)$.

To obtain a better algorithm, we modify Algorithm $\mathcal{B}$ and design a new black-box that contains one more processor but delays the output stream only 1 round after the input stream. We call this modified black-box Algorithm $\mathcal{C}$. This algorithm broadcasts $m > 1$ messages from an external source processor to 2^k internal processors, for $k \geq 2$. Algorithm $\mathcal{C}$ requires $m+k+2$ rounds to broadcast m messages to $n-1 = 2^k$ internal processors.

Finally, using the new black-box provided by Algorithm $\mathcal{C}$, we are able to build an optimal-time algorithm (to within an additive term of 2) for any values of m and n. We call this algorithm Algorithm $\mathcal{D}$. This algorithm concatenates instances of decreasing sizes of Algorithm $\mathcal{C}$'s black-boxes, such that the delays introduced by all the black-boxes in the sequence are shadowed by the delay of the first black-box. Algorithm $\mathcal{D}$ requires at most $m + \lceil \log n \rceil$ rounds.

3 Description of the algorithm

Algorithm $\mathcal{A}$

This algorithm broadcasts $m = 1$ messages among $n = 2^k$ processors in $k = \log n$ rounds. The algorithm consists of doubling the number of processors that know the message in each round. Following is a formal description of Algorithm $\mathcal{A}$ that is used in Algorithms $\mathcal{B}$ and $\mathcal{C}$. We denote the $n = 2^k$ processors using the 2^k binary addresses $(\epsilon_k, \ldots, \epsilon_1)$, where $\epsilon_j \in \{0, 1\}$ for $1 \leq j \leq k$. Let the broadcast originator be processor $s = (0, \ldots, 0)$, and let M be the broadcast message. In round 1 of the algorithm, processor s sends M to processor $(0, \ldots, 0, 1)$. In general, in round i of the algorithm, for $1 \leq i \leq k$, processor $(0, \ldots, 0, \epsilon_{i-1}, \ldots, \epsilon_1)$ sends M to processor $(0, \ldots, 0, 1, \epsilon_{i-1}, \ldots, \epsilon_1)$. One may use induction to verify that at the end of round i, for $1 \leq i \leq k$, all the processors of the form $(0, \ldots, 0, \epsilon_i, \ldots, \epsilon_1)$, where $\epsilon_j \in \{0, 1\}$ for $1 \leq j \leq i$, indeed know message M. From this description, it is clear that the algorithm broadcasts message M to all $n = 2^k$ processors in $k = \log n$ rounds.

A simple modification of this description is required in case the broadcast originator is not processor $(0, \ldots, 0)$. If processor $s = (\sigma_k, \ldots, \sigma_2, \sigma_1)$ is the broadcast originator, then in round 1 processor s sends message M to processor $(\sigma_k, \ldots, \sigma_2, 1 - \sigma_1)$. In general, in round i, for $1 \leq i \leq k$, each processor p that knows message M sends it to the processor whose address is obtained by complementing the ith bit of the address of p. That is, a processor that knows message M and whose address is $(\delta_k, \ldots, \delta_{i+1}, \delta_i, \delta_{i-1}, \ldots, \delta_1)$ sends message M in round i to processor $(\delta_k, \ldots, \delta_{i+1}, 1 - \delta_i, \delta_{i-1}, \ldots, \delta_1)$.

In this description, the order in which processors complement their address bits is not crucial, as long as all processors follow the same order in a synchronized manner. In particular, the algorithm works correctly if processors start by complementing their jth bit, for some $1 \leq j \leq k$, and cyclically complement bits $j+1, j+2 \ldots, j+k-1$ (where the index arithmetic is done in a cycle of k). This is the case since there is no significance to the relative order of processors' address bits. We use the notation $\mathcal{A}(s, M, j)$ to indicate an instance of Algorithm $\mathcal{A}$, where the broadcast originator is processor s, the broadcast message is M, and processors start by first complementing the jth bit of their addresses and then cyclically complement bits $j+1$, $j+2$, until the $j-1$ bit.

Algorithm $\mathcal{B}$

This algorithm broadcasts $m > 1$ messages among $n = 2^k$ processors in $m + k = m + \log n$ rounds. Informally, the broadcast originator (root) sends the m messages one after the other in a cyclic manner to $k = \log n$ designated processors. Each of these k processors applies Algorithm $\mathcal{A}$ among all the n processors, thereby acting as a broadcast originator for each message it receives from the root. Consecutive applications of Algorithm $\mathcal{A}$ by one designated processor do not overlap, since Algorithm $\mathcal{A}$ completes in exactly k rounds and the designated processor receives a new message exactly once every k rounds. However, different applications of Algorithm $\mathcal{A}$ by two designated processors may overlap and may introduce conflicts for non-designated processors. Following is a description of Algorithm $\mathcal{B}$ that avoids any such conflicts.

We denote the $n = 2^k$ processors, as before, by the 2^k binary addresses $(\epsilon_k, \ldots, \epsilon_1)$, where $\epsilon_j \in \{0, 1\}$ for $1 \leq j \leq k$. Let the broadcast originator be processor $s = (0, \ldots, 0)$, and let $M_1, \ldots, M_m$ be the sequence of m messages to broadcast. In round i, for $1 \leq i \leq m$, the broadcast originator s sends message M_i to processor $p_j = (0, \ldots, 0, 1, 0, \ldots, 0)$, where the 1-bit in the address of processor p_j appears in the jth index position for $j = ((i-1) \bmod k) + 1$. Processor p_j receives message M_i from the root in round i and applies Algorithm $\mathcal{A}(p_j, M_i, j+1)$ for k rounds starting in round $i+1$. Notice that processor p_j does not need to return message M_i to the root s in round $i + k$ as instructed by Algorithm $\mathcal{A}(p_j, M_i, j + 1)$.

The correctness of the send/receive schedules of this algorithm follows from the fact that the n processors can be thought of being logically arranged as a k-dimensional hypercube where communication occurs only along the hypercube edges. In round i, each processor p may send a message to and receive a message from a processor q if and only if the addresses of p and q differ only in the bit in the jth position, for $j = ((i-1) \bmod k)) + 1$. This implies that there are no send/receive conflicts since messages are always sent and received along a distinct dimension of the k-dimensional hypercube.

Algorithm $\mathcal{C}$

This algorithm broadcasts $m > 1$ messages from an external source processor to $n = 2^k$ internal processors, for $k \geq 2$, in $m + k + 2$ rounds. In addition, this algorithm delays by one round the stream of m messages injected by the external source. That is, when considering the set of $n = 2^k$ internal processors as a black-box, the set absorbs a stream of messages and emits the same stream one round later. This black-box behavior of the $n = 2^k$ internal processors is exploited in Algorithm $\mathcal{D}$.

The process of broadcasting the m messages from the external source to the $n = 2^k$ internal processors, for $k \geq 2$, is a modification of Algorithm $\mathcal{B}$. Informally, the external source sends the m messages one after the other in a cyclic manner to $k = \log n$ designated processors. Each of these designated processors, upon receiving a message from the external source, first sends that message outside the set and then follows Algorithm $\mathcal{A}$ to broadcast that message among $2^k - 1$ out of the $n = 2^k$ internal processors. (One of the 2^k internal processors has a special role which we explain later.) Many of these $2^k - 1$ internal processors behave as in Algorithm $\mathcal{B}$. However, applying Algorithm $\mathcal{B}$ without any changes would result in receive-conflicts for the k designated processors. We resolve these receive-conflicts by using the extra internal processor. We next describe this modification of Algorithm $\mathcal{B}$ into Algorithm $\mathcal{C}$ more formally.

The system consists of one external source processor and $n = 2^k$ internal processors, for $k \geq 2$. Denote one of the internal processors by ψ, and use the k-bit binary addresses to denote the external source processor and the remaining $2^k - 1$ internal processors. Let the external source processor be $s = (0, \ldots, 0)$, and let $M_1, \ldots, M_m$ be the sequence of m messages to broadcast. The external source s sends message M_i in round i to processor $p_j = (0, \ldots, 0, 1, 0, \ldots, 0)$, where the 1-bit in the address of processor p_j appears in the jth index position for $j = ((i-1) \bmod k) + 1$. Processor p_j receives message M_i in round i and sends it outside the set in round $i+1$. Starting in round $i+2$, processor p_j applies algorithm $\mathcal{A}(p_j, M_i, j+1)$ for k rounds. However, processor p_j does not return message M_i to the external source processor in round $i + k + 1$, as instructed by Algorithm $\mathcal{A}(p_j, M_i, j + 1)$.

This modification of Algorithm $\mathcal{B}$ results in having only one round of delay between the input stream of messages into the set of $n = 2^k$ internal processors and the output stream of messages from the set. However, this modification also causes receiving conflicts for the k designated processors. Since, in Algorithm $\mathcal{C}$, each designated processor p_j starts algorithm $\mathcal{A}$ in the hypercube one round later than is required by Algorithm $\mathcal{B}$, such processor p_j is scheduled to receive a message from some internal hypercube processor in the same round in which it receives a message from the external source s. To circumvent this conflict, the special processor ψ is used. The rule regarding processor ψ is as follows. Assume that an internal processor q is instructed to send a message, according to Algorithm $\mathcal{A}$, to some designated processor p_j in the same round that p_j gets a new message from the external source s. Then, instead, processor q sends that message to processor ψ, and in the following round processor ψ forwards this message to processor p_j. Notice that processor p_j is indeed available to receive a message from processor ψ immediately following the round in which it receives a new message from the external source s, since in this round processor p_j sends a message outside the set and is not receiving a message from any internal hypercube processor.

Algorithm $\mathcal{D}$

This algorithm broadcasts $m > 1$ messages among n processors in at most $m + \lceil \log n \rceil$ rounds, for any values of m and n. If n is a power of 2, then Algorithm $\mathcal{B}$ can be used to produce a running time of $m - 1 + \log n$ rounds Otherwise, a chain of black-boxes of decreasing sizes, each implementing Algorithm $\mathcal{C}$, is used. Each black-box contains a number of processors that is a power of 2, such that the total number of processors in all the black-boxes is $n - 1$. The output stream of each black-box serves as the input stream to the next black-box in the chain. The input stream to the first black-box is supplied by the broadcast originator and the output stream of the last black-box is discarded. Following is a more formal description of Algorithm $\mathcal{D}$ for the case where n not a power of 2.

Assume that n is not a power of 2. Let $n = 1 + (\sum_{i=1}^{\ell} 2^{k_i}) + r$, where $k_1 > k_2 > \cdots > k_\ell \geq 2$ and $0 \leq r \leq 3$. A different treatment is given to the broadcast originator, to the main $n - 1 - r$ processors, and to the remaining r processors for $0 \leq r \leq 3$. The $n - 1 - r$ processors are partitioned into ℓ disjoint sets, $S_1, \ldots, S_\ell$, where set S_i contains 2^{k_i} processors, for $1 \leq i \leq \ell$. The broadcast originator is put into set S_0, and the r remaining processors are put into set $S_{\ell+1}$. Each of the ℓ sets $S_1, \ldots, S_\ell$ implements Algorithm $\mathcal{C}$ internally. The broadcast originator in set S_0 sends the stream of m messages to set S_1; the output stream of messages from set S_1 serves as the input stream of messages to set S_2; and so on until set S_ℓ, which sends its output stream to set $S_{\ell+1}$. Inside set $S_{\ell+1}$, the r remaining processors, for $0 \leq r \leq 3$, are cascaded such that the stream of m messages is passed from one processor to another. One may verify that since the broadcasting process works well in each one of the sets S_i, then the concatenation of the sets S_i produces a broadcasting algorithm of m messages among n processors.

The analysis of the running time of Algorithm $\mathcal{D}$ is as follows. Assume that the broadcast originator in set S_0 starts sending the stream of m messages in round 1. Then, set S_i, for $1 \leq i \leq \ell$, starts receiving the stream and starts applying Algorithm $\mathcal{C}$ in round i. This implies that the broadcasting process in set S_i is completed in round $m + i + k_i + 1$. However, since k_i is a strictly decreasing sequence, it follows that $1 + k_1 \geq i + k_i$ for all values of $1 \leq i \leq \ell$. Consequently, all the processors in sets $S_0, \ldots, S_\ell$ know the m messages by round $m + k_1 + 2$. Now, Since n is not a power of 2, it follows that $k_1 = \lceil \log n \rceil - 1$ and, therefore, all the $n - r$ processors in sets $S_0, \ldots, S_\ell$ know all the m messages by round $m + \lceil \log n \rceil + 1$. It is now only necessary to consider the r, $0 < r \leq 3$, remaining processors in set $S_{\ell+1}$. These processors start receiving the stream of the m messages in round $\ell + 1$, and, therefore, they know all the m messages at most by round $(\ell + 1) + (m - 1) + 2 = \ell + m + 2$. Now, because k_i is a strictly decreasing sequence, because $k_1 = \lceil \log n \rceil - 1$, and because $k_\ell \geq 2$, it must be that $\ell \leq \lceil \log n \rceil - 2$. This shows that the r processors in set $S_{\ell+1}$ also know all the m messages by round $\ell + m + 2 \leq m + \lfloor \log n \rfloor$. This analysis proves that the time complexity of Algorithm $\mathcal{D}$ is at most $m + \lceil \log n \rceil + 1$ rounds.

References

[1] A. Bar-Noy and S. Kipnis, "Designing broadcasting algorithms in the postal model for message-passing systems," *4th ACM Symp. on Parallel Algorithms and Architectures,* pp. 11–22, 1992.

[2] A. Bar-Noy and S. Kipnis, "Multiple message broadcasting in the postal model," *7th International Parallel Processing Symp.,* IEEE, Newport Beach, CA, 1993.

[3] J. Beetem, M. Denneau, and D. Weingarten, "The GF-11 supercomputer," *Proc. of the International Symposium on Computer Architecture,* pp. 108–115, 1985.

[4] J. Bruck, R. Cypher, and C. T. Ho, "Multiple message broadcasting with generalized Fibonacci trees," *4th Symp. on Parallel and Distributed Processing,* IEEE, 1992.

[5] D. Clark, B. Davie, D. Farber, I. Gopal, B. Kadaba, D. Sincoskie, J. Smith, and D. Tennenhouse, "The AURORA gigabit testbed," *Computer Networks and ISDN,* 1991.

[6] I. Cidon and I. Gopal, "PARIS: an approach to integrated high-speed private networks," *International Journal of Digital and Analog Cabled Systems,* Vol. 1, pp. 77–85, 1988.

[7] E. Cockayne and A. Thomason, "Optimal multi-message broadcasting in complete graphs," *11th SE Conference on Combinatorics, Graph Theory, and Computing,* pp. 181–199, 1980.

[8] W. J. Dally, A. Chien, S. Fiske, W. Horwat, J. Keen, M. Larivee, R. Lethin, P. Nuth, S. Wills, P. Carrick, and G. Fyler "The J-Machine: a fine-grain concurrent computer," *Information Processing 89,* Elsevier Science Publishers, IFIP, pp. 1147–1153, 1989.

[9] A. M. Farley, "Broadcast time in communication networks," *SIAM Journal on Applied Math.,* Vol. 39, pp. 385–390, 1980.

[10] C. T. Ho, "Optimal broadcasting on SIMD hypercubes without indirect addressing capability," *Journal on Parallel and Distributed Computing,* Vol. 13, pp. 246–255, 1991.

[11] C. E. Leiserson, Z. S. Abuhamdeh, D. C. Douglas, C. R. Feynman, M. N. Ganmukhi, J. V. Hill, W. D. Hillis, B. C. Kuszmaul, M. A. St. Pierre, D. S. Wells, M. C. Wong, S.-W. Yang, and R. Zak, "The network architecture of the Connection Machine CM-5," *4th ACM Symp. on Parallel Algorithms and Architectures,* ACM, pp. 272–285, 1992.